Independent component analysis

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

In this notebook, we exemplify **independent component analysis** (ICA) by applying it to an audio file that mimics the cocktail-party problem.

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import Audio
from scipy.io import wavfile
from sklearn.decomposition import PCA, FastICA

plt.style.use("ggplot")
```

We will load and play a short audio file (4 seconds), which was created by mixing two clips of the speech track from one of the lecture videos. This file is **in stereo** (two channels) with **sampling frequency** 44.1 kHz (i.e., 44100 samples per second) and **bit depth** 16 (i.e., $2^{16} = 65536$ possible levels). The two speech tracks are placed slightly apart in the stereo field. This creates a linear mixture that conforms exactly to the key assumption of ICA.

In [2]:
```
rate, data = wavfile.read("Mix-for-ICA.wav")
print("Sampling frequency:", rate, "Hz")
print("Data shape:", data.shape)
print("Data value type:", data.dtype)
```

```
Sampling frequency: 44100 Hz
Data shape: (176400, 2)
Data value type: int16
```

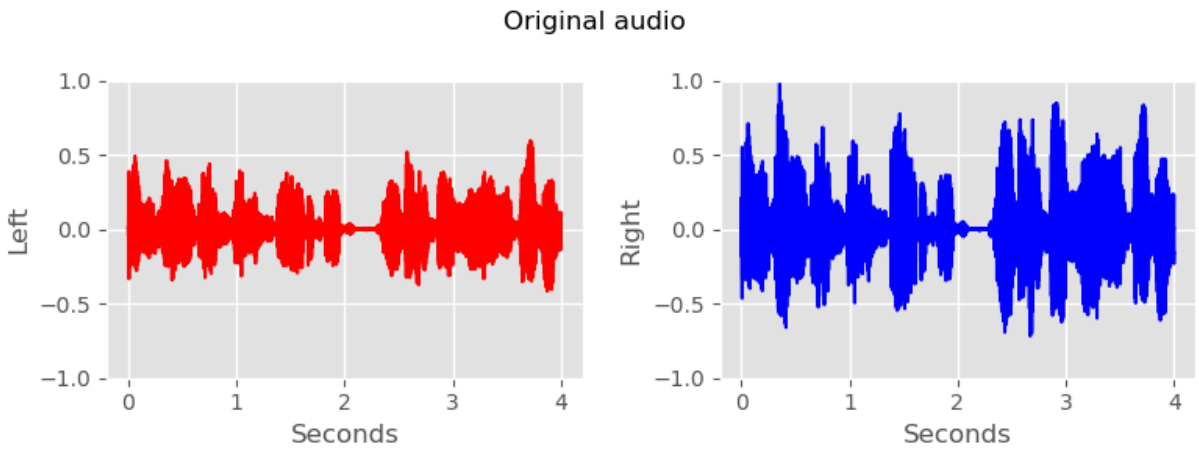
```
/var/folders/mt/gq61qprx3xn80cyfh7v7vmzm0000gp/T/ipykernel_14172/1728291936.py:1: WavFileWarnin
g: Chunk (non-data) not understood, skipping it.
  rate, data = wavfile.read("Mix-for-ICA.wav")
```

In [3]:
```
# Convert from 16-bit fixed point (np.int16) to 64-bit floating point (np.float64) and
# normalise values to range [-1, 1] (32768 = 2^16 / 2 is the largest possible value):
data = data.astype(np.float64) / 32768
data = data.T  # IPython.display.Audio expects shape (2, n_samples) for stereo audio
Audio(data=data, rate=rate)
```

Out[3]:

Let us also plot the waveforms of both left and right channels:

In [4]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 3))

grid = np.arange(0, data.shape[1]) / rate
axs[0].plot(grid, data[0, :], color="red")
axs[0].set_xlabel("Seconds")
axs[0].set_ylabel("Left")
axs[0].set_ylim(-1.0, 1.0)
axs[1].plot(grid, data[1, :], color="blue")
axs[1].set_xlabel("Seconds")
axs[1].set_ylabel("Right")
axs[1].set_ylim(-1.0, 1.0)

fig.suptitle("Original audio")
plt.tight_layout()
plt.show()
```

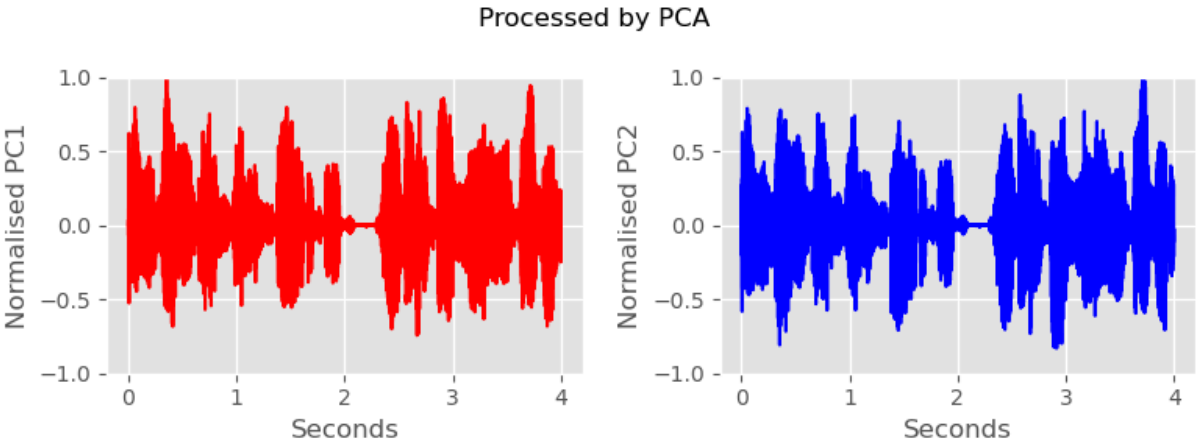


We will first try to separate the speech tracks using **PCA**. After applying PCA, we normalise the channels:

In [5]:
```
pca = PCA()
data_pca = pca.fit_transform(data.T)
data_pca[:, 0] = data_pca[:, 0] / np.max(data_pca[:, 0])
data_pca[:, 1] = data_pca[:, 1] / np.max(data_pca[:, 1])
data_pca = data_pca.T
```

First plot them:

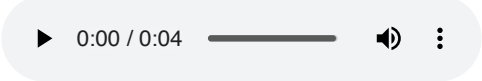
In [6]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 3))

axs[0].plot(grid, data_pca[0, :], color="red")
axs[0].set_xlabel("Seconds")
axs[0].set_ylabel("Normalised PC1")
axs[0].set_ylim(-1.0, 1.0)
axs[1].plot(grid, data_pca[1, :], color="blue")
axs[1].set_xlabel("Seconds")
axs[1].set_ylabel("Normalised PC2")
axs[1].set_ylim(-1.0, 1.0)

fig.suptitle("Processed by PCA")
plt.tight_layout()
plt.show()
```



And then play them component by component. First PC1:

In [7]:
```
Audio(data=data_pca[0, :], rate=rate)
```

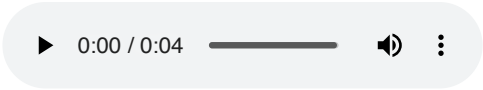
Out[7]:



Then PC2:

In [8]:
```
Audio(data=data_pca[1, :], rate=rate)
```

Out[8]:

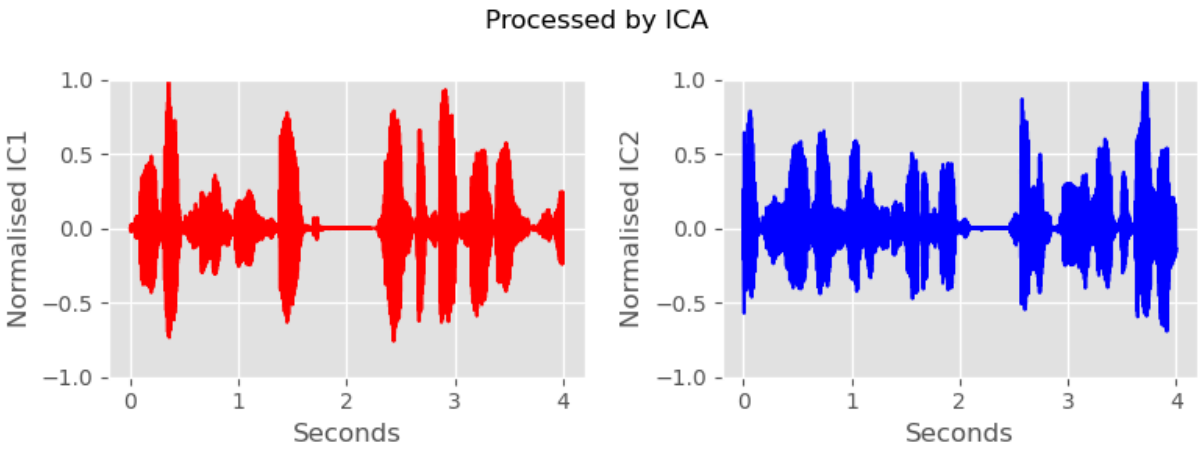


It is clear that PCA is unable to separate the signals. We will then try **ICA**. The class `FastICA` from the module `sklearn.decomposition` performs the version of ICA seen in lectures. We normalise the resulting components (since their scaling is arbitrary anyway).

In [9]:
```
ica = FastICA(whiten="unit-variance", random_state=12345)
data_ica = ica.fit_transform(data.T)
data_ica[:, 0] = data_ica[:, 0] / np.max(data_ica[:, 0])
data_ica[:, 1] = data_ica[:, 1] / np.max(data_ica[:, 1])
data_ica = data_ica.T
```

Let us plot the components:

In [10]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 3))

axs[0].plot(grid, data_ica[0, :], color="red")
axs[0].set_xlabel("Seconds")
axs[0].set_ylabel("Normalised IC1")
axs[0].set_ylim(-1.0, 1.0)
axs[1].plot(grid, data_ica[1, :], color="blue")
axs[1].set_xlabel("Seconds")
axs[1].set_ylabel("Normalised IC2")
axs[1].set_ylim(-1.0, 1.0)

fig.suptitle("Processed by ICA")
plt.tight_layout()
plt.show()
```



They look reassuringly dissimilar. Let us play IC1:

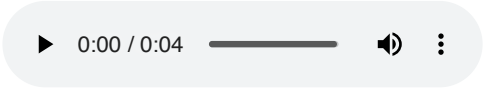
In [11]:
```
Audio(data=data_ica[0, :], rate=rate)
```

Out[11]:

And then IC2:

In [12]:
```
Audio(data=data_ica[1, :], rate=rate)
```

Out[12]:



The separation is remarkably clear and artifact-free.